

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	M PRITHIYANKA SHREE	Reg. No.:	192521144
Section / Batch:	2025	Date:	23-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

An internet service provider must route data packets across a large network efficiently to minimize latency. Analyze how shortest path algorithms like Dijkstra or Bellman-Ford can be applied. Identify constraints such as network congestion and dynamic edge weights. Design a solution for real-time routing optimization. Discuss scalability for large networks. Evaluate time complexity and memory usage. Interpret performance in high-speed communication systems.

Code of Conduct

I PRITHIYANKA SHREE M (192521144) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: PRITHIYANKA SHREE M

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Your Code

```
1 n = int(input())
2
3 graph = []
4
5 for i in range(n):
6     row = list(map(int, input().split()))
7     graph.append(row)
8
9 source = int(input())
10
11 INF = 999999
12
13 distance = []
14 visited = []
15
16 for i in range(n):
17     distance.append(INF)
18     visited.append(False)
19
20 distance[source] = 0
21
22 # Dijkstra Algorithm
23 for count in range(n):
24     u = -1
25
26     # Find the unvisited node with minimum distance
27     for i in range(n):
28         if visited[i] == False:
29             if u == -1 or distance[i] < distance[u]:
30                 u = i
31
32     if u == -1:
33         break
34
35     visited[u] = True
36
37     # Update distances
38     for v in range(n):
39         if graph[u][v] != 0 and visited[v] == False:
40             new_distance = distance[u] + graph[u][v]
41
42             if new_distance < distance[v]:
43                 distance[v] = new_distance
44
45 # Print shortest distances
46 for i in range(n):
47     print("Node", i, ":", distance[i])
```

INPUT AND OUTPUT:

Input

```
4
0 5 0 10
0 0 3 0
0 0 0 1
```

Output

```
Node 0 : 0
Node 1 : 5
Node 2 : 8
Node 3 : 9
```